

LEVEL 28

Chapter 26: Cheat Sheet & Decision Matrices

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 26: The Pre-Interview Cheat Sheet & Decision Matrices

← [Chapter 25: Study Plans](#) · [Index](#) · Next → [Chapter 27: Question Bank](#)

This is the one chapter designed to be re-read the morning of an interview. Everything here is a compressed pointer back into the full chapters — if anything here doesn't immediately make sense, that's the signal to go back and re-read that chapter, not to memorize this page in isolation.

26.1 The Compact Cheat Sheet

Requirements — Always ask: functional scope (what exactly are we building), scale (DAU/RPS), latency needs, consistency needs, and whether this is read-heavy or write-heavy. Budget 3-5 min. (*Ch.14, Step 1*)

Capacity — $\text{DAU} \rightarrow \text{RPS} = (\text{DAU} \times \text{actions/day}) / 86,400$. Peak = avg $\times$ 2-3 (normal) or $\times$ 5-10 (flash event). Storage = $\text{records/day} \times \text{size} \times \text{retention} \times \text{replication}(3x)$. Round aggressively, say your rounding out loud. (*Ch.3*)

APIs — 4-6 core endpoints, method + path + key params. Mention idempotency keys on unsafe writes. Cursor-paginate anything list-shaped and growing. (*Ch.9*)

Database — SQL by default unless you have a specific reason not to (multi-record ACID, relational integrity). Document DB for varying schema. Wide-column/DynamoDB for extreme write scale + simple access patterns. Redis for pure key lookups. (*Ch.4.6, decision matrix below*)

Cache — Cache-aside is the default pattern. Always state a TTL and an eviction policy (LRU is the default-default). Name the three failure modes (stampede/penetration/avalanche) if caching is a deep-dive area. (*Ch.4.5*)

Queue — Reach for one when: work can be decoupled from the user-facing request, a burst needs absorbing, or multiple independent consumers need the same event. Kafka for replay/multi-consumer-group streaming; SQS for simple point-to-point; SNS for fan-out. (*Ch.7*)

Scaling — Stateless app layer scales horizontally, trivially. DB scaling ladder: single DB $\rightarrow$ read replica $\rightarrow$ cache $\rightarrow$ shard $\rightarrow$ distributed DB. Justify each step with a number from your capacity estimate, never jump straight to sharding. (*Ch.5.5*)

Consistency — State it per data type, not once for the whole system. Strong for money/inventory/booking-critical data. Eventual for feeds/counts/search/analytics. Name the CAP/PACELC trade-off explicitly when it's relevant. (*Ch.2.5, Ch.6*)

Reliability — Timeouts everywhere, retries with backoff+jitter, circuit breakers on cross-service calls, replication (Multi-AZ minimum), and a stated RPO/RTO if disaster recovery comes up. (Ch.6.1-6.3, Ch.11.4)

Security — Weave in naturally: auth at the gateway, encryption in transit always / at rest for sensitive data, rate limiting, and PCI-scope minimization for anything payment-adjacent. Don't save it all for the end as an afterthought bullet. (Ch.10)

Observability — Logs + metrics + traces, correlation IDs, and an SLI/SLO if reliability is a deep-dive topic. This is your strength — use it. (Ch.11)

Trade-offs — End every major decision with "the trade-off here is..." State it even when not asked. This is the single highest-leverage habit in the entire roadmap. (Ch.14.1, Step 9)

26.2 Decision Matrices

SQL vs. NoSQL

Need	Choose	Why
Multi-record ACID transactions, relational integrity	SQL (PostgreSQL/MySQL)	Enforced constraints, joins, mature tooling
Rapidly varying schema per record	Document (MongoDB)	Flexible schema, fetch-whole-document access pattern
Extreme write throughput, simple known access patterns	Wide-column (Cassandra) or DynamoDB	Built for this exact shape, horizontally native
Unsure / genuinely relational, moderate scale	SQL	Safer, more battle-tested default

MongoDB vs. DynamoDB (a common follow-up to the above)

Need	Choose	Why
Self-managed flexibility, complex document queries/aggregation	MongoDB	Richer query language, more query flexibility
Zero operational overhead, predictable single-digits latency at any scale, simple key-based access	DynamoDB	Fully managed, auto-scaling, AWS-native
Multi-cloud/on-prem requirement	MongoDB (Atlas or self-hosted)	Not locked to AWS

Kafka vs. RabbitMQ

Need	Choose	Why
High-throughput streaming, replay, multiple independent consumer groups	Kafka	Log-based, retains data, natively multi-consumer
Complex conditional routing (topic wildcards, fan-out rules)	RabbitMQ	Exchange types built exactly for this
Simple task distribution, lower ops overhead	RabbitMQ (or SQS if on AWS)	Simpler operational model

Kafka vs. SQS

Need	Choose	Why
Replay, ordered partitions, multiple independent consumers at different paces	Kafka / Kinesis	Log-retention model
Simple point-to-point work queue, fully managed, minimal setup	SQS	Purpose-built simplicity, no infra to run

Redis vs. Memcached

Need	Choose	Why
Rich data structures, persistence option, built-in HA/clustering	Redis	Default choice for almost everything today
Pure, maximally memory-efficient key-value cache, no persistence needed, extreme scale	Memcached	Simpler, multi-threaded natively, smaller memory overhead per key

REST vs. gRPC vs. GraphQL

Need	Choose	Why
Public/partner-facing API, cacheable, universal client support	REST	Ubiquitous, HTTP-cache-friendly
Internal service-to-service, low latency, streaming	gRPC	Binary protobuf, HTTP/2 multiplexing, generated stubs
Client-driven data shape, aggregating multiple sources, avoiding over/under-fetching	GraphQL	One flexible query, client controls the shape

Synchronous vs. Asynchronous Communication

Need	Choose	Why
Caller needs an immediate answer to proceed	Synchronous (REST/gRPC)	User/caller is blocked on the result regardless
Action doesn't need to block the caller; callee downtime shouldn't fail the caller	Asynchronous (queue/event)	Decouples availability and timing between services

Monolith vs. Microservices

Need	Choose	Why
New product, unclear domain boundaries, small team	Modular monolith	Boundaries aren't proven yet; avoid paying distributed-systems tax prematurely
Proven independent scaling needs, multiple teams needing independent deploys, real operational maturity	Microservices	Independent scaling/deployment justifies the real complexity cost

ECS vs. EKS

Need	Choose	Why
Simpler AWS-native container orchestration, tighter IAM/Fargate integration	ECS	Less operational overhead, smaller learning curve
Multi-cloud portability, full Kubernetes ecosystem (Helm, CRDs, operators), existing K8s expertise	EKS	Portability and ecosystem breadth

Single-Region vs. Multi-Region

Need	Choose	Why
Standard availability needs, cost-sensitivity, simpler operations	Single-region + Multi-AZ	Multi-AZ alone covers most real failure scenarios at far lower complexity
Whole-region-outage protection genuinely justified, or serving latency-sensitive users across distant geographies	Multi-region	Real business case required — not a default; carries genuine consistency/latency/cost complexity

General-Purpose Technology Selection Table

Requirement	Technology	Why
Low-latency cache	Redis	Sub-ms, rich structures, TTL support

Requirement	Technology	Why
Large object/blob storage	S3	Durable, scalable, cheap, offloads bytes from compute
Async event streaming with replay	Kafka	Log-based, multi-consumer, ordered per partition
Simple managed queue	SQS	Zero ops, point-to-point
Strong relational transactions	PostgreSQL / MySQL	ACID, joins, mature
Massive write scale, simple access pattern	Cassandra / DynamoDB	Horizontally native, write-optimized
Full-text search with ranking	Elasticsearch	Inverted index, relevance scoring, aggregations
Real-time bidirectional communication	WebSocket	Persistent, full-duplex
Real-time server-push-only	SSE	Simpler than WebSocket for one-directional streams

Next → [Chapter 27: Question Bank](#) — 270+ practice questions across beginner, intermediate, advanced, fintech, real-time, e-commerce, and distributed-systems categories.